

Lab 3 - Introduction to GPU programming with CUDA

Oguz Kaya
oguz.kaya@universite-paris-saclay.fr

Environment. We use the CUDA version of Compiler Explorer, <https://cuda.godbolt.org>, which compiles and runs a program on a real GPU: language **CUDA C++**, compiler **NVCC**; in the compiler pane, open the “Add new...” menu and choose “Executor From This”; in the executor pane that appears, set the compiler option **-O2**. The executor shows the output of the program. Save your code regularly in a local file. On a machine with an NVIDIA GPU and the CUDA toolkit, the same files compile and run with

```
nvcc -O2 program.cu -o program then ./program
```

Submission. For the questions that ask for a comment or a result, write your answers as comments at the top of the corresponding source file, in the block provided for this purpose; the source files are the only thing you submit.

GPU programming with CUDA. In this lab, we write our first GPU programs, using the minimal toolbox of the lecture: a *kernel* is the program of one thread; a *launch* runs it on $B \times T$ threads, in B blocks of T ; a thread knows its index among all the threads of the launch; the CPU waits for the GPU with `cudaDeviceSynchronize`.

<code>__global__ void f(...) { ...}</code>	a kernel: the program of one thread, run on the GPU
<code>f<<B, T>>(...);</code>	launch: B blocks of T threads, $B \times T$ threads in all, each executes <code>f</code>
<code>blockIdx.x * blockDim.x + threadIdx.x</code>	my index i among all the threads of the launch, from 0 to $BT - 1$
<code>gridDim.x * blockDim.x</code>	the number p of threads of the launch ($B \times T$)
<code>cudaDeviceSynchronize()</code>	the CPU waits until the GPU is done

These are the only CUDA constructs you need and are allowed to use in this lab. The GPU has its own memory: the arrays a kernel works on live there, and the data travels between the two memories by copies. This part is *given* in the skeleton codes, and commented: `cudaMalloc` makes room for an array in the GPU memory, `cudaMemcpy` copies an array from one memory to the other, `cudaFree` frees the room. A launch returns at once, so `cudaDeviceSynchronize()` is needed before the program ends, and before reading the clock when timing a kernel.

Remember the two rules for variables: a variable declared *inside* the kernel belongs to the thread that runs it (one copy per thread); the GPU sees *none* of the CPU's variables: an array must be a GPU array, passed to the kernel as a pointer, and a scalar is passed as an argument, by value (every thread gets a copy). And the one rule for correctness: in a launch, **two threads must never write the same cell**.

Part 1

Hello world with GPU threads

The goal of this exercise is to run a kernel on many threads and to see what each of them does. A skeleton code is provided in the file `hello.cu`; it contains the launch, with B blocks of T threads.

Ex. 1

- a) Write the kernel `hello` in which each thread prints one line with three pairs of values:

```
thread <t>/<T> of block <b>/<B>, global thread <i>/<p>
```

`<t>` is its number inside its block and `<T>` the number of threads of a block; `<b>` the number of its block and `<B>` the number of blocks; `<i>` its global index among all the threads of the launch, computed from the other four, and `<p>` the total number of threads.

- b) Run the program with 64 threads in all, split as $B \times T = 64 \times 1, 32 \times 2, 16 \times 4, 8 \times 8, 4 \times 16, 2 \times 32$ and 1×64 . How do the six values change from one configuration to the next, and which do not change?

Saxpy on the GPU

Saxpy is the operation $y \leftarrow a \cdot x + y$ on two vectors x and y of N numbers and a scalar a : $y[i] \leftarrow a \cdot x[i] + y[i]$ for every i . A skeleton code is provided in the file `saxpy.cu`; it allocates the vectors `x` and `y` of $N = 10^6$ single precision numbers on the CPU (4 MB per array), with `x[i] = i` and `y[i] = 1`, their GPU twins `dx` and `dy`, and contains the copies between the two memories, the timing of the kernel, and the verification of the result (as well as a first, empty launch: the first launch of a program pays a start-up cost, which is thus kept out of the timings).

Ex. 2

- a) Write the kernel `saxpy` in which each thread handles *one* element, exactly as a PU of the PRAM: its index i is `blockIdx.x * blockDim.x + threadIdx.x`, and it computes $y[i] \leftarrow a \cdot x[i] + y[i]$. The launch creates $BT \geq N$ threads, so the last block may contain threads that have no element: the kernel must start with the guard `if (i < n)`.
- b) Set $T = 256$ threads per block, compute the number of blocks $B = \lceil N/T \rceil$ so that the threads cover the whole array, and launch the kernel; the skeleton times it and checks the result.
- c) Run the program with $T = 1, 2, 4, 8, \dots, 1024$ threads per block, B being adjusted each time so that the whole array is covered. Make sure that the result is correct in every configuration, and observe how the time of the kernel changes.
- d) The skeleton times the kernel; time the two copies as well, the copy in (CPU $\rightarrow$ GPU, before the kernel) and the copy out (GPU $\rightarrow$ CPU, after it), with `now()` in the same way, and compare their times with the time of the kernel (with $T = 256$): how many times longer is a copy than the kernel? What does this mean for a program that launches a single kernel, and for a program that launches many kernels on the same data?